

Praktikum 1 Pengenalan Pengujian Perangkat Lunak



Dosen Pengampu:

Lukman Hakim S.Kom.

Disusun Oleh:

Farel Indra Syahputra

362358302171

POLITEKNIK NEGERI BANYUWANGI
JURUSAN TEKNIK BISNIS DAN INFOMATIKA
2025/2026

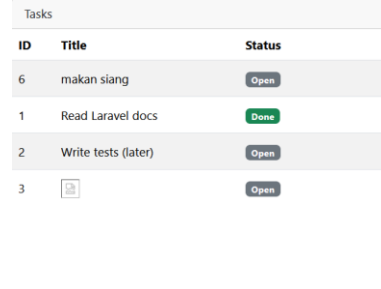
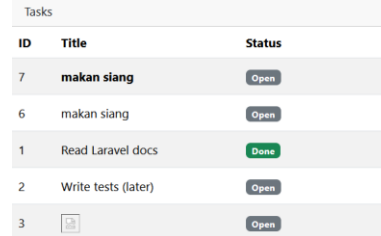
1. Tujuan Praktikum

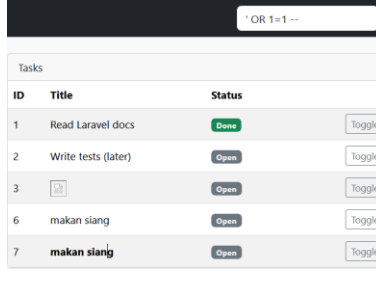
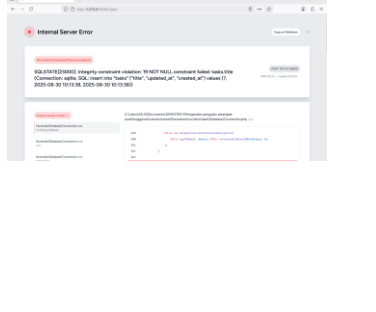
- Mengenali dan menganalisis kelemahan kode dalam aplikasi web.
- Melatih kemampuan debugging, secure coding, dan best practice Laravel.
- Memperbaiki bug terkait validasi, keamanan, dan logika aplikasi.

Langkah Praktikum

Kerjakan eksperimen berikut, lalu catat temuan Anda.

1. **Input HTML/Script ke form**
 - Tambahkan task dengan judul `<b>tes</b>` atau `<script>alert(1)</script>`.
 - Amati hasil tampilan.
 - Apa yang terjadi? Bug apa ini?
2. **Coba query search aneh**
 - Gunakan fitur *search* dengan input ' OR 1=1 --.
 - Apa dampaknya pada query SQL?
3. **Hapus task lewat URL langsung**
 - Klik tombol **Delete**.
 - Perhatikan URL yang digunakan.
 - Apakah aman jika destructive action dilakukan dengan **GET**?
4. **Cari tahu apakah ada bug lain dalam aplikasi tersebut**

No	Deskripsi pengujian	Langkah Pengujian	Data Input	Expected Output	Actual Output	Status
1	Menguji input data task dengan data normal	Masukkan data pada isian task Klik tombol save		Data task makan siang masuk ke tabel task	Data task makan siang masuk ke tabel task	Pass
2	Uji input HTML	Tambahkan task dengan <code></code>		Teks tampil literal tanpa format	Teks tampil tebal (HTML dieksekusi)	Fail

3	Uji SQL Injection di fitur Search	Masukkan ' OR 1=1 -- di search box		Query gagal atau ditolak (tidak semua data muncul)	Semua data muncul karena query injeksi berhasil	Fail
4	Uji hapus task lewat URL langsung	Klik tombol Delete lalu perhatikan URL → akses ulang URL http://127.0.0.1:8000/tasks/delete/1		Aksi delete hanya boleh dengan POST + CSRF	ask langsung terhapus hanya dengan GET request	Fail
5	Uji input kosong pada form task	Klik Save tanpa isi Task		Tampil Pesan error	Aplikasi melempar error SQL (NOT NULL constraint failed) bukannya menampilkan pesan error validasi.	Fail

1. Link CSS

Sebelum

```
<link href=("/css/custom.css)" rel="stylesheet">
```

Sesudah :

```
<link href="{{ asset('css/custom.css') }}" rel="stylesheet">
```

2. Route Delete

Sebelum :

```
Route::get('/tasks/delete/{id}', [TaskController::class, 'destroy'])->name('tasks.destroy.get');
```

Sesudah :

```
Route::delete('/tasks/{id}', [TaskController::class, 'destroy'])->name('tasks.destroy');
```

3. Search (hindari SQL Injection)

Sebelum :

```
public function search(Request $request)
{
```

```

$q = $request->query('q', "");
$results = [];

if ($q !== "") {

    $results = Task::whereRaw("title LIKE '%{$q}%'")
        ->orderBy('created_at', 'desc')
        ->get();
}

return view('tasks.index', ['tasks' => $results, 'q' => $q, 'search' => true]);
}

```

Sesudah :

```

public function search(Request $request)
{
    $q = $request->query('q', "");
    $results = [];

    if ($q !== "") {
        $results = Task::where('title', 'like', "%{$q}%")
            ->orderBy('created_at', 'desc')
            ->get();
    }

    return view('tasks.index', ['tasks' => $results, 'q' => $q, 'search' => true]);
}

```

4. Delete di blade

Sebelum :

```

<a href="{{ route('tasks.destroy.get', $task->id) }}" class="btn btn-sm btn-outline-danger">Delete</a>

```

Sesudah :

```

<form action="{{ route('tasks.destroy', $task->id) }}" method="POST" class="d-inline">
    @csrf
    @method('DELETE')
    <button type="submit" class="btn btn-sm btn-outline-danger">Delete</button>
</form>

```

5. Validasi input

Sebelum :

```

public function store(Request $request)
{

    Task::create($request->all());
    return redirect()->route('tasks.index');
}

```

```
}
```

Sesudah :

```
public function store(Request $request)
{
    $request->validate([
        'title' => 'required|string|max:255',
    ]);

    Task::create([
        'title' => $request->title,
    ]);

    return redirect()->route('tasks.index')->with('success', 'Task berhasil ditambahkan!');
}
```

Lalu Untuk menampilkan pesan error :

Sebelum :

```
<div class="mb-3">
    <label class="form-label">Title</label>
    <input type="text" name="title" class="form-control" placeholder="e.g., Fix bugs">
</div>
```

Sesudah :

```
<div class="mb-3">
    <label class="form-label">Title</label>
    <input
        type="text"
        name="title"
        class="form-control @error('title') is-invalid @enderror"
        placeholder="e.g., Fix bugs"
        value="{{ old('title') }}"
    >

    @error('title')
        <div class="invalid-feedback">
            {{ $message }}
        </div>
    @enderror
</div>
```

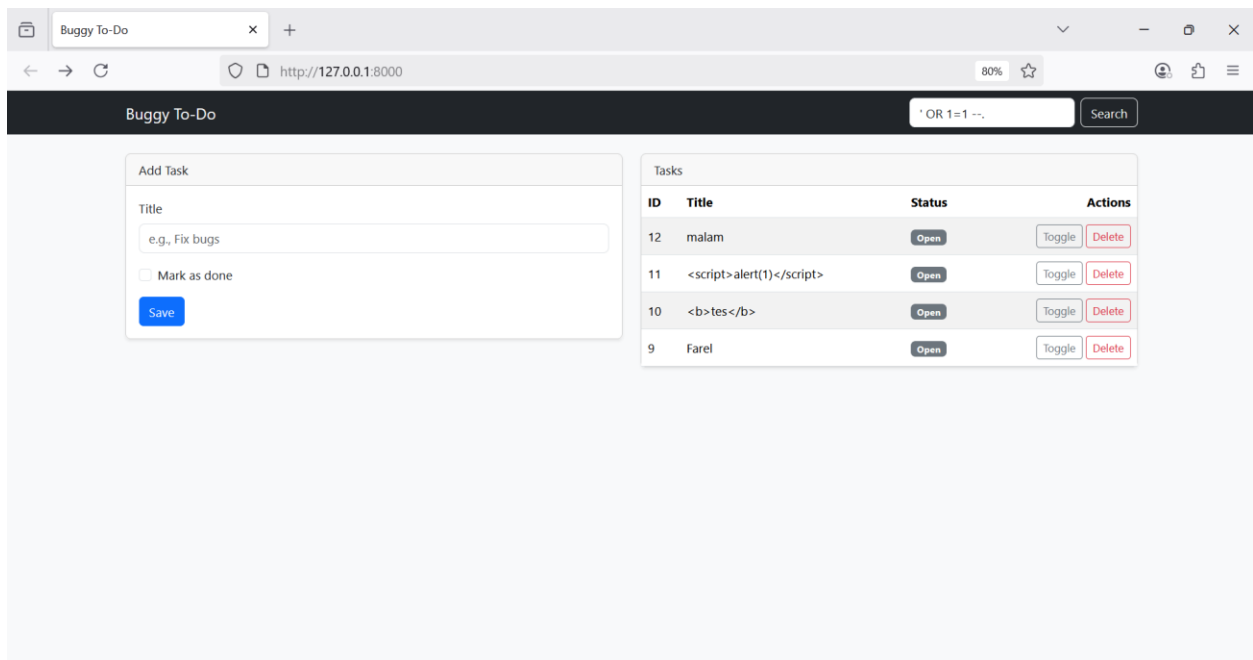
Kesimpulan :

Dari praktikum disimpulkan:

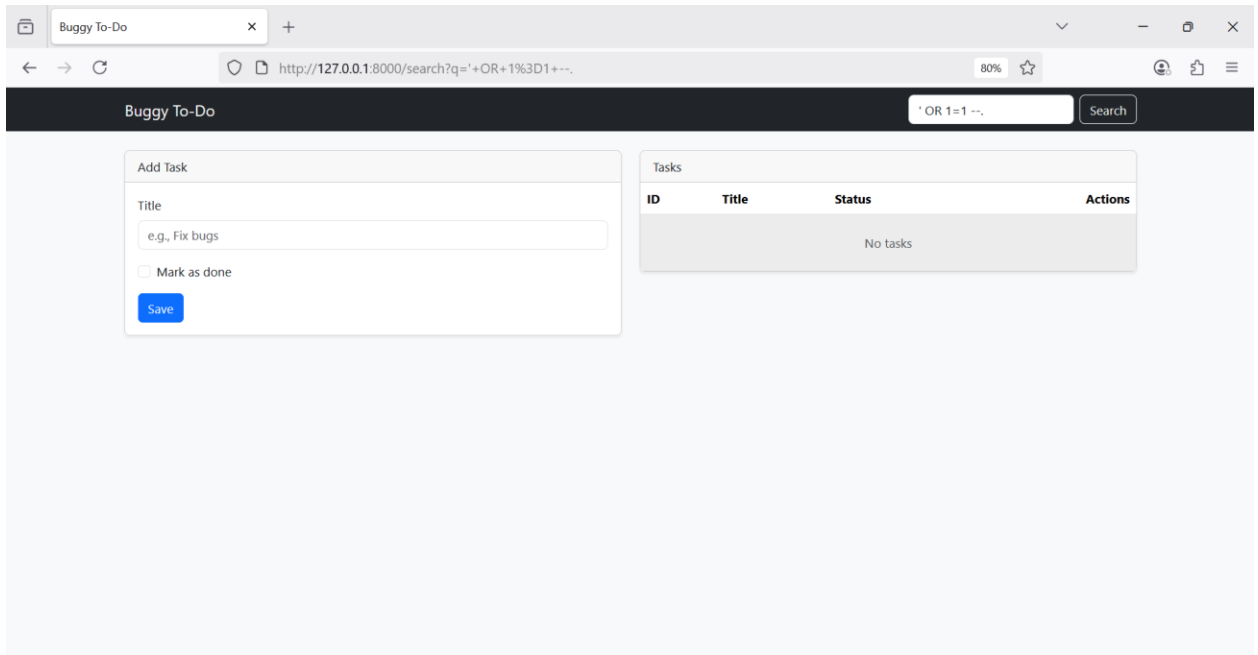
- Aplikasi awal memiliki beberapa kelemahan: XSS, SQL Injection, delete via GET, dan tidak adanya validasi input.
- Setelah perbaikan, aplikasi menjadi lebih aman dengan:
 - Escape input HTML otomatis oleh Blade.
 - Query builder Eloquent untuk mencegah SQL Injection.
 - Proteksi CSRF + Method Spoofing agar delete tidak bisa dilakukan via URLGET.
 - Validasi input Laravel untuk mencegah error SQL.
- Dengan demikian, aplikasi sudah sesuai dengan secure coding practice di Laravel.

Screenshoot akhir :

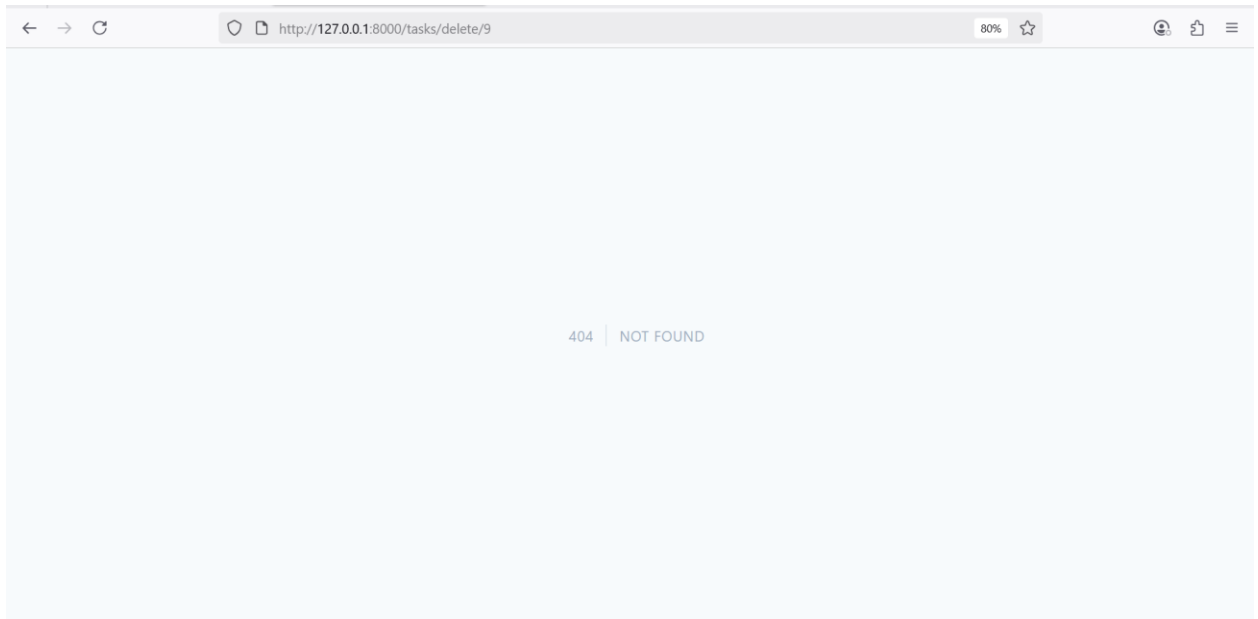
1



2



3.



4

